학습 노트 (6). HLS 예제 검증과 내 칩 설계의 시작

오픈소스 반도체 설계 및 제작 종합설계 (2026 Fall)

공과대학 기계공학부 안준영　학번 2024-11078　2026 년 11 월 5 일

들어가며

이번 주에는 고위합성(HLS) 예제 두 개를 돌려서 C++ 기준 모델과 Gemini 가 만든 RTL 을 비교했다. 이어서 지난주 노트 끝에 적어 둔 2 차 델타-시그마 변조기를 직접 C++로 쓰고 RTL 로 옮겨서 같은 방식으로 검증했다. 두 작업에서 같은 문제가 서로 다른 모습으로 나왔다. C++에서는 함수를 부르면 바로 값이 나오지만, RTL 에서는 그 값이 언제 나오는지를 따로 정해야 한다는 점이다.

1. 실습 환경

x86 리눅스에 Verilator 5.030 을 소스로 빌드해서 썼고, SystemC 는 그동안 쓰던 3.0.2 를 그대로 썼다. FIR 예제의 잡음 생성기(cnoise.cpp)는 GSL 라이브러리의 FFT 함수를 쓰는데, 이번 환경에는 GSL 을 설치할 수 없었다. 그래서 cnoise.cpp 가 부르는 함수 여섯 개만 같은 이름과 인자로 직접 짜서 넣었다. 잡음을 만드는 데에만 쓰이는 부분이라 비교 결과에는 영향이 없다.

2. 시계 예제

Clock.cpp 는 부를 때마다 1 초를 더하는 함수이고, 함수 안의 static 변수 세 개가 시, 분, 초를 기억한다. Gemini 는 함수 호출 한 번을 클럭 한 번으로, static 변수를 레지스터로 옮겼다. 테스트벤치는 매 클럭 C++ 함수와 RTL 의 출력을 비교하고, 1,000 번째 클럭 근처에서 약 100 클럭 동안 start 를 내려 시계를 멈춘다.

저장소의 RTL 은 출력을 assign 으로 바로 내보내고, 90,002 줄이 모두 OK 였다. 맨 끝에 주석으로 남아 있는 블록을 살려서 Gemini 가 처음 준 것처럼 출력을 레지스터에 한 번 더 담게 하니 OK 104 줄, ERROR 89,898 줄이 나왔다. OK 가 나온 곳을 찾아보니 0~4 번 클럭(리셋 구간)과 1,007~1,105 번 클럭(시계가 멈춘 구간)뿐이었다. 값이 변하지 않을 때만 맞는다는 것은 계산은 맞고 출력이 한 클럭 늦다는 뜻이다. 어느 쪽이 맞는지는 C++ 코드만 보고는 정할 수 없다. C++ 함수에는 값이 언제 나와야 하는지에 대한 정보가 없기 때문이다.

3. FIR 필터 예제

3.1 다섯 가지 RTL

fir8.cpp 는 계수 4, 12, 25, 34, 34, 25, 12, 4 의 8 탭 필터이고, 출력은 곱의 합에 64 를 더해 7 비트 오른쪽으로 민 값이다. Gemini 는 이것을 병렬, 직렬, RAM 사용, 곱셈기 없는 구조 두 가지로 모두 다섯 가지 RTL 로 만들었다. 테스트벤치는 입력 하나를 넣고 out_valid 가 올라올 때까지 기다리는 핸드셰이크로 4,800 개를 넣는다.
OUTPUT_LATENCY=1, FIX_SATURATION=YES 로 먼저 돌린 결과는 TABLE 1 과 같다.

TABLE 1. FIR RTL 다섯 가지의 비교 결과 (입력 4,800 개)

RTL	처음 결과	확인한 것
0 병렬	Pass 49	OUTPUT_LATENCY 를 4 로 두면 4,800 개 모두 Pass. 2, 3, 5 는 모두 실패
1 직렬	Pass 641	end begin 으로 else 가 빠져 있다(동료 노트와 같은 원인)
2 RAM	Pass 197	원인 분석은 동료 노트 참고
3 곱셈기 없음	Pass 1,439	원인 분석은 동료 노트 참고
4 곱셈기 없음(FF)	빌드 실패	negedge 가 megedge 로 오타. 고치면 4,800 개 모두 Pass

1~3 번은 김예원과 이수빈이 원인을 찾아 고친 과정을 6 주차 노트에 자세히 적어 두어서 따로 고치지는 않았다. 처음 결과가 틀리는 모습은 두 사람의 노트와 같았다. 여기서는 동료 노트에 없던 세 가지를 적는다.

3.2 0 번의 out_valid

0 번은 지연 값만 맞추면 모든 출력이 맞는다. 그런데 왜 4 인지를 따라가 보니 인터페이스 문제였다. 시프트 레지스터, 곱 레지스터, 합 레지스터, 출력 레지스터가 모두 in_valid 가 1 일 때만 움직인다. 그래서 파이프라인은 새 입력이 들어와야만 한 칸 밀린다. 반면 out_valid 는 in_valid 를 한 클럭 늦춘 것이라, 입력을 넣고 한 클럭 뒤에 무조건 1 이 된다. 그때 y 에 나와 있는 값은 방금 넣은 입력이 아니라 세 번 전에 넣은 입력의 결과다. out_valid 는 1 인데 y 는 이번 입력의 답이 아닌 셈이다. 테스트벤치가 OUTPUT_LATENCY 로 기준 값을 늦춰서 맞춘 것이고, 이것은 테스트벤치가 DUT 안의 구조를 알고 있어야 쓸 수 있는 방법이다. 핸드셰이크를 둔 이유가 바깥에서 안을 몰라도 쓸 수 있게 하는 것이라면 0 번의 out_valid 는 그 역할을 못 하고 있다.

3.3 포화 처리

0 번을 OUTPUT_LATENCY=4, FIX_SATURATION=NO 로 다시 돌리면 4,800 개 중 18 개가 틀린다. 계수의 합이 150 이라 이득이 150/128, 약 1.17 배이고, 큰 입력이 이어지면 (합+64)>>7 이 255 를 넘는다. 이때 C++ 모델은 8 비트로 넘쳐서 작은 값이 되고, RTL 은 255 로 자른다. 16 비트 출력 모드로 C++ 모델만 돌려 본 다른 실행에서는 23 개(0.5 퍼센트)가 255 를 넘었다. 코드는 둘 다 각자 맞게 짜여 있고, 넘쳤을 때 어떻게 할지에 대한 약속이 서로 달랐다.

3.4 테스트 입력의 랩어라운드

입력 신호를 보다가 이상한 점을 찾았다. 테스트벤치는 잡음(0~102)에 진폭이 다른 정현파 네 개를 더해 8 비트 data_t 에 넣는다. 정현파 네 개의 최댓값은 31, 63, 127, 127 이고 잡음까지 더하면 최대 450 이 될 수 있다. 같은 식으로 4,800 개를 만들어 세어 보니 1,684 개(35.1 퍼센트)가 255 를 넘었다. 평균은 223.6, 최댓값은 434 였다. 이 값들은 8 비트로 잘리면서 400 이 144 로, 300 이 44 로 바뀐다(Fig. 1a). 스펙트럼으로 보면 넣은 정현파 주파수 말고도 곳곳에 선이 생기고, 바닥이 약 20dB 올라간다(Fig. 1b).

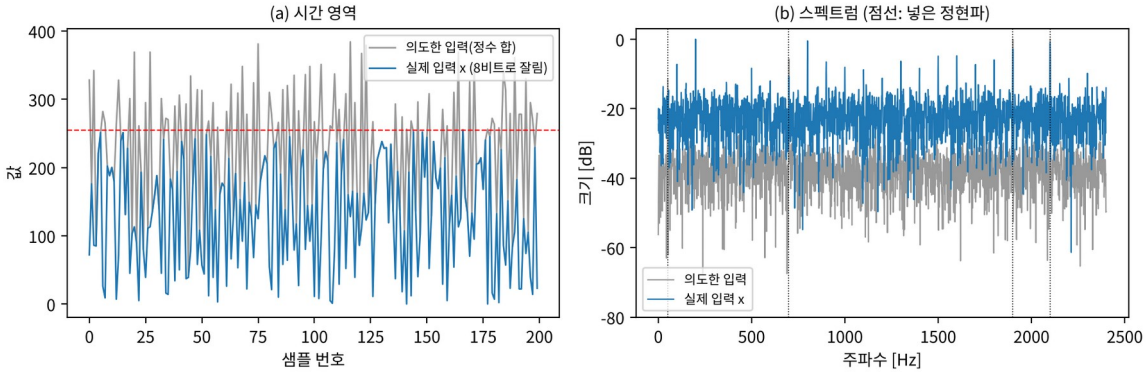


Fig. 1. FIR 테스트벤치의 입력. (a) 정수로 더한 값(회색)과 8 비트로 잘린 실제 입력(파랑), (b) 두 입력의 스펙트럼.

그런데도 0 번과 4 번은 4,800 개가 모두 Pass 다. C++ 모델과 RTL 이 똑같이 잘린 입력을 받기 때문이다. 비트 단위 비교는 두 구현이 같은지는 알려 주지만, 넣은 시험 신호가 의도한 것인지는 알려 주지 않는다. 필터가 1,000Hz 아래만 통과시키는지 보려면 정현파의 진폭을 줄여서 합이 255 를 넘지 않게 해야 한다고 생각한다.

4. 내 칩 설계의 시작: 2 차 델타-시그마 변조기

지난주 노트에 수중 백스캐터 노드의 PDM 변조를 전용 회로로 만들어 보고 싶다고 적었다. 그 첫 단계로 16 비트 PCM 표본을 받아 1 비트 PDM 을 내보내는 2 차 델타-시그마 변조기를 만들었다. 순서는 HLS 예제와 같게 했다. C++로 비트 정확 모델을 먼저 쓰고, AI 에게 RTL 로 옮기게 한 뒤, SystemC 테스트벤치에서 C++ 모델과 비트 단위로 비교했다. PDM 이 제대로 나왔는지 보려고 PDM 을 다시 PCM 으로 바꾸는 3 차 CIC 데시메이터도 같은 방식으로 만들었다.

4.1 C++ 기준 모델

PDM 클럭은 PDM 마이크에서 흔히 쓰는 3.072MHz 로 두었다. MCU 는 96kHz 로만 표본을 넘겨주고, 변조기는 같은 표본을 32 클럭 동안 쓴다. 변조기는 적분기 두 개와 비교기 하나다. 직전 출력이 1 이면 +32,768, 0 이면 -32,768 을 되먹임해서 첫 적분기는 s1 += x - fb, 둘째 적분기는 s2 += s1 - fb 를 계산하고, s2 가 0 이상이면 1 을 낸다. 1 kHz(진폭 0.40)와 7 kHz(0.15) 두 정현파를 넣었을 때 |s1|은 최대 72,884, |s2|는 136,244 까지 커졌다. 부호를 포함해 19 비트면 담을 수 있는 크기라서, RTL 에서는 여유를 두고 24 비트로 잡았다.

4.2 RTL 두 가지

처음 RTL(v1)은 적분기를 레지스터로 두고 비블로킹 대입(<=)으로 썼다. RTL 에서는 자연스러운 모양이지만, 이렇게 쓰면 s2 는 이번 클럭에 갱신된 s1 이 아니라 갱신 전의 s1 을 더한다. C++에서는 s1 을 먼저 바꾸고 그 값을 바로 s2 에 쓰기 때문에 계산 순서가 달라진다. 두 번째 RTL(v2)은 이번 클럭에 들어갈 새 s1 을 조합 회로로 먼저 만들고, 그 값을 s2 계산에 바로 쓰게 했다.

PDM 76,800 비트를 비교한 결과, v2 는 두 클럭 늦게 나오는 것만 빼면 C++ 모델과 모든 비트가 같았다. v1 은 어떻게 맞춰 봐도 절반 정도가 달랐다. 출력 비트를 CIC 로 다시 PCM 으로 바꿔 보니 차이가 분명했다(Fig. 3). v2 는 입력 파형을 그대로 되살렸고 0~20 kHz 의 신호 대 잡음비가 56.3dB 였다. v1 은 처음 0.5ms 정도는 입력을 따라가다가

그 뒤로 무너졌고 신호 대 잡음비는 -0.3dB 였다. 비트 폭 제한 없이 v1 의 계산을 다시 해 보니 s2 가 87,418,060 까지 커졌다. 비트 폭이 모자라서가 아니라 루프 자체가 발산한 것이다.

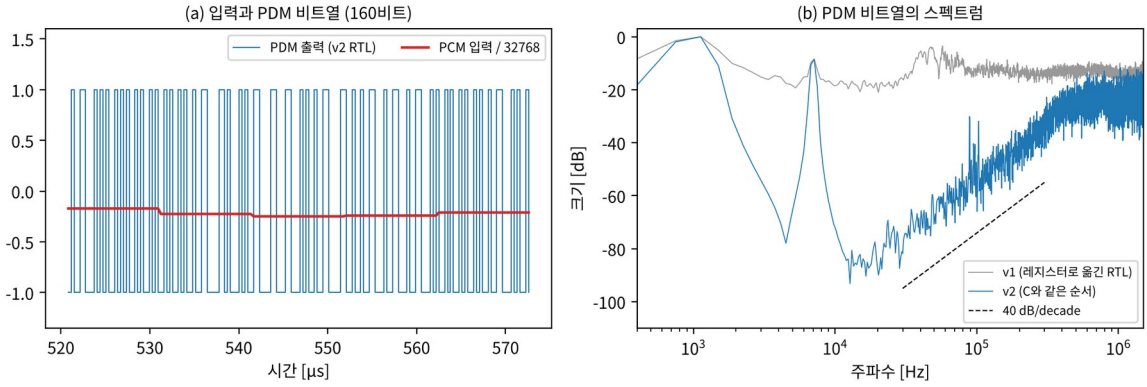


Fig. 2. 변조기 RTL 의 출력. (a) PCM 입력과 v2 가 낸 PDM 비트열, (b) PDM 비트열의 스펙트럼. v2 는 신호 대역의 잡음이 눌리고 높은 주파수로 갈수록 약 40dB/decade 로 커진다.

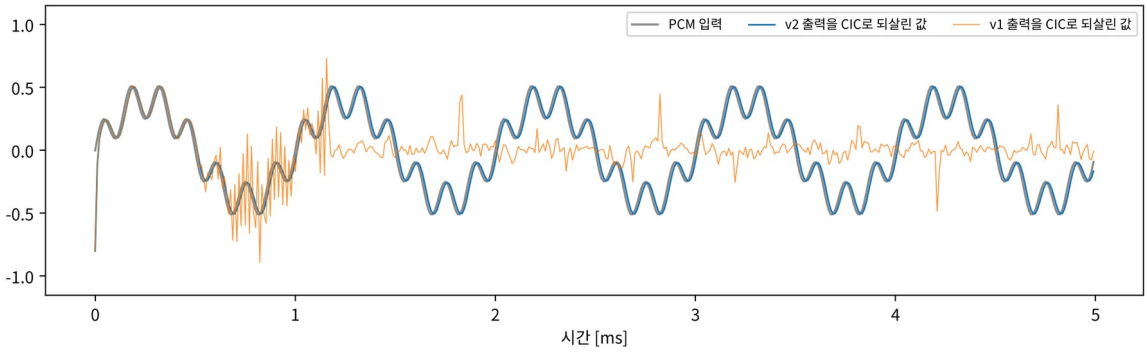


Fig. 3. 두 변조기의 PDM 출력을 CIC 데시메이터로 다시 PCM 으로 바꾼 결과. v1 은 약 0.5ms 뒤에 발산한다.

Fig. 2b 에서 v2 의 PDM 스펙트럼은 신호 대역에서 잡음이 낮고 주파수가 높아질수록 약 40dB/decade 로 올라간다. 2 차 변조기라 잡음을 높은 주파수 쪽으로 밀어내는 모양이 이렇게 나와야 한다. v1 은 이 모양이 없다. 레지스터를 쓰면 루프 안에 지연이 하나 더 들어가고, 되먹임 루프의 전달함수가 바뀐다. 그 바뀐 루프가 안정하지 않았던 것이다.

4.3 CIC 데시메이터에서는 괜찮았던 같은 차이

CIC 데시메이터도 똑같이 두 가지로 만들었다. 적분기 세 개는 PDM 클럭마다, 미분기 세 개는 32 클럭마다 계산하고, 내부 폭은 3 x log2(32) + 1 = 16 비트다. 레지스터로 옮긴 v1 은 출력 2,432 개 중 2,363 개가 C++ 모델과 달랐다. 그런데 C++ 모델의 입력을 3 클럭 늦춰서 넣으니 2,432 개가 모두 같았다. 적분기 세 단이 각각 한 클럭씩 늦게 더하는 것뿐이라, 값은 같고 시점만 3 클럭(약 1μs) 밀린 것이다. v2 는 처음부터 2,432 개가 모두 같았고, 끝에 1 만 넣는 구간에서 출력이 최댓값 32,768 에 닿아 16 비트로 넘치지 않는 것도 확인했다.

TABLE 2. 직접 만든 RTL 네 가지의 검증 결과

설계	RTL	C++ 모델과 비교	판단
변조기	v1 레지스터	PDM 비트의 약 50 퍼센트 다름	발산. 신호 대 잡음비 -0.3dB
변조기	v2 순서 맞춤	76,800 비트 모두 같음	신호 대 잡음비 56.3dB
CIC	v1 레지스터	2,432 개 중 2,363 개 다름	입력을 3 클럭 늦춘 모델과 모두 같음. 정상
CIC	v2 순서 맞춤	2,432 개 모두 같음	최댓값 32,768 확인

같은 방식으로 옮겼는데 CIC 에서는 시점만 바뀌고, 변조기에서는 회로가 망가졌다. 차이는 되먹임이 있는지였다. 앞으로만 흐르는 구조에서는 지연이 생겨도 전체가 늦어질 뿐이지만, 되먹임 루프 안에 생긴 지연은 루프의 성질을 바꾼다. 그리고 비교 결과의 숫자만으로는 이 둘을 구분할 수 없었다. CIC v1 의 불일치 2,363 개는 문제가 아니었고, FIR 0 번의 Pass 4,800 개는 out_valid 의 문제를 가리고 있었다. 틀렸다는 결과가 나오면 설계가 틀렸는지, 시점만 다른지, 테스트벤치의 약속이 다른지부터 가려야 했다.

4.4 크기와 핀

변조기 v2 의 플립플롭은 24 비트 적분기 두 개와 출력 1 비트로 49 개이고, 적분기를 19 비트로 줄이면 39 개가 된다. CIC 는 16 비트 레지스터 여섯 개와 카운터, 출력을 합쳐 118 개다. 이수빈은 노트에서 FIR PE 가 850 셀에 코어 0.51mm² 였으니 28 핀 프레임에는 1,000 셀 안팎이 한계로 보인다고 적었다. 변조기는 그보다 훨씬 작을 것으로 보이지만 아직 합성은 해 보지 않았다. 핀은 더 빠듯하다. 16 비트 입력을 병렬로 받으면 입력만 16 핀이고, 클럭, 리셋, 출력, 전원까지 더하면 28 핀이 거의 찬다. MCU 에서 표본을 직렬로 받는 방식을 같이 생각해 봐야 한다.

5. 상사가 주말에 앱을 만들었다는 글을 읽고

IEEE Spectrum 에 실린 Brian Jenney 의 글은 동작하는 데모가 실제 일의 복잡함을 가린다고 말한다. 이번 주에 비슷한 것을 직접 봤다. 변조기 v1 은 처음 0.5ms 동안 입력을 잘 따라갔다. 그 구간만 화면에 띄웠다면 동작한다고 생각했을 것이다. FIR 0 번도 지연 값 하나만 맞추면 4,800 개가 모두 Pass 였다. 이번에 RTL 을 짜는 데는 몇 분이 걸리지 않았고, 시간은 대부분 결과가 왜 그렇게 나왔는지 확인하는 데 들었다. 글쓴이가 엔지니어에게 AI 모델이 어떻게 동작하는지 배우라고 한 것처럼, 회로를 만드는 쪽에서는 AI 가 짠 코드가 언제 틀리는지 알아보는 능력이 필요하다고 느꼈다.

6. 남은 의문

- 변조기 v2 는 한 클럭 안에서 덧셈과 뺄셈 네 번을 이어서 한 뒤 부호를 보고, CIC v2 는 덧셈과 뺄셈 여섯 번을 이어서 한다. 3.072MHz 면 한 클럭이 약 326ns 인데, 0.5µm 표준 셀로 합성하면 이 경로가 시간 안에 들어오는지 타이밍 검사로 확인해야 한다.
- CIC v1 처럼 시점만 다른 경우에는 RTL 을 C++에 맞추기보다 C++ 모델을 하드웨어에 맞추는 편이 회로가 단순하다. 기준 모델을 어느 쪽에 맞출지는 무엇을 기준으로 정해야 할까.
- FIR 테스트벤치는 잡음 시드로 현재 시각을 써서 실행할 때마다 입력이 달라진다. 틀린 경우를 다시 보려면 시드를 고정하는 선택지가 있으면 좋겠다.

- 변조기를 칩으로 만든다면 입력을 어떻게 받을지(병렬, SPI 같은 직렬, 칩 안에서 파형 생성)를 핀 수와 MCU 가 깨어 있는 시간을 같이 놓고 정해야 한다.

마치며

HLS 예제에서는 AI 가 만든 RTL 을 검증했고, 내 설계에서는 내가 만든 C++ 모델을 기준으로 AI 가 옮긴 RTL 을 검증했다. 두 경우 모두 RTL 을 만드는 일보다 그 RTL 이 언제, 어떤 값을 내야 하는지를 정하는 일이 더 어려웠다. 다음에는 변조기와 CIC 를 ETRI 표준 셀로 합성해서 셀 수와 타이밍을 확인하고, 7 주차의 설계 계획서에 반영하려고 한다.